

CSE 4403: Algorithms

Lecture04

Graph - 1

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

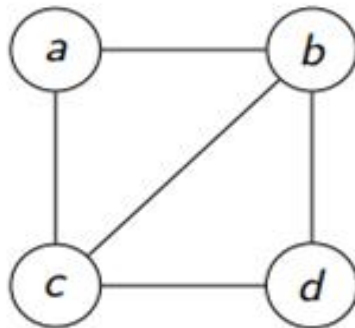
Email: anikafarzana@iut-dhaka.edu

Graph

- A Graph is composed of a set of vertices(**V**) and a set of edges(**E**). The vertices are connected with each other through edges.

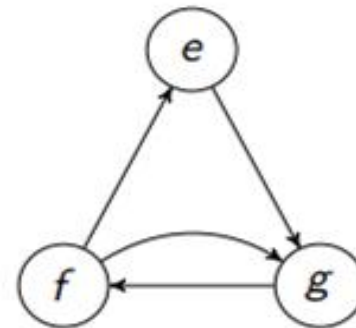
■ Graph, $G = (V, E)$

- V = Set of vertices/nodes → Denoted by circles
- E = Set of edges → Denoted by lines/arrows
 - ▶ Undirected: $e = \{u, v\}$ → Unordered pairs
 - ▶ Directed: $e = (u, v)$ → Ordered pairs



$$V = \{a, b, c, d\}$$

$$E = \{\{a,b\}, \{a,c\}, \{b,c\}, \{b,d\}, \{c,d\}\}$$



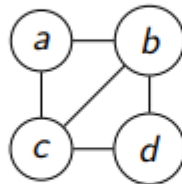
$$V = \{e, f, g\}$$

$$E = \{(e,g), (f,e), (f,g), (g,f)\}$$

Adjacency Matrix

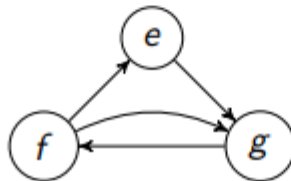
■ Adjacency Matrix

- Array $adjM$ of size $|V| \times |V|$
- Each element $\in \{0, 1\}$
- (Undirected) For vertices $u, v \in V$, $adjM[u][v] = \begin{cases} 1 & \text{if } \{u, v\} \in E \\ 0 & \text{o/w} \end{cases}$



	a	b	c	d
a	0	1	1	0
b	1	0	1	1
c	1	1	0	1
d	0	1	1	0

- (Directed) For vertices $u, v \in V$, $adjM[u][v] = \begin{cases} 1 & \text{if } (u, v) \in E \\ 0 & \text{o/w} \end{cases}$

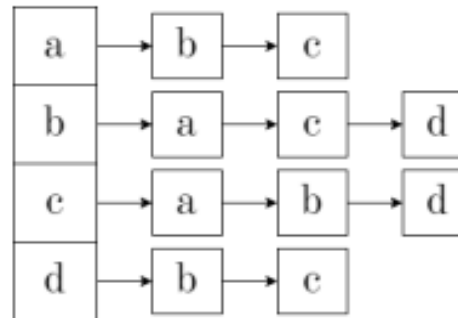
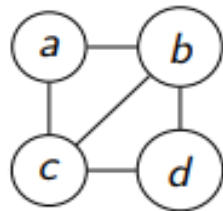


	e	f	g
e	0	0	1
f	1	0	1
g	0	1	0

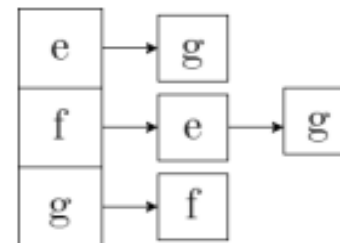
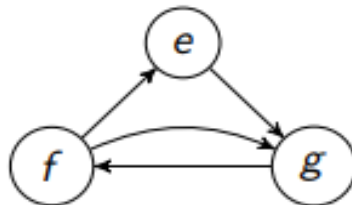
Adjacency List

■ Adjacency List

- Array `adjL` of size $|V|$
- Each element contains a linked list
- (Undirected) For each vertex $u \in V$, $adjL[u] = \{v \in V | \{u, v\} \in E\}$



- (Directed) For each vertex $u \in V$, $adjL[u] = \{v \in V | (u, v) \in E\}$



Graph Traversal : BFS

Adjacency List Implementation

```
6  void bfs(vector<int> graph[], int start) {
7      bool visited[100] = {false};
8      queue<int> q;
9
10     visited[start] = true;
11     q.push(start);
12
13     while (!q.empty()) {
14         int current = q.front();
15         q.pop();
16
17         cout << current << " ";
18
19         for (int k = 0; k < graph[current].size(); k++){
20             int neighbor = graph[current][k];
21             if (!visited[neighbor]) {
22                 visited[neighbor] = true;
23                 q.push(neighbor);
24             }
25         }
26     }
27 }
```

Time complexity

Adjacency List

- A node is popped from the queue.
- In the inner loop, only the non-visited neighbors of the current node are pushed. So, inner loop runs a total of E times, where E is the total no. of edges in the graph.
- Since each node is pushed only once in the queue, the other lines of the outer loop runs a total of V times.
- So, time complexity = $O(V) + O(E) = O(V + E)$.

Graph Traversal : BFS

Adjacency Matrix Implementation

```
29 void bfs(int graph[][100], int vertices, int start) {
30     bool visited[100] = {false};
31     queue<int> q;
32
33     visited[start] = true;
34     q.push(start);
35
36     while (!q.empty()) {
37         int current = q.front();
38         q.pop();
39
40         cout << current << " ";
41
42         for (int i = 0; i < vertices; i++) {
43             if (graph[current][i] && !visited[i]) {
44                 visited[i] = true;
45                 q.push(i);
46             }
47         }
48     }
49 }
```

Time complexity

Adjacency Matrix

- A node is popped from queue.
- In the inner loop, all of the V vertices in the current node's row of the adjacency matrix are checked. If they are its neighbor and not visited, they are pushed in the queue.
- It repeats for every vertex pushed in queue. Since, only non-visited nodes are pushed, outer loop runs for V times.
- So, time complexity = $O(V) * V = O(V^2)$.

Graph Traversal : DFS

Adjacency Matrix Implementation

```
5  void dfs(vector<vector<int>> &adj, vector<bool> &visited, int node) {
6      visited[node] = true;
7      cout << node << " ";
8
9      for (int i : adj[node]) {
10         if (!visited[i]) {
11             dfs(adj, visited, i);
12         }
13     }
14 }
15
16 void dfs_caller(vector<vector<int>> &adj, int start) {
17     vector<bool> visited(adj.size(), false);
18     dfs(adj, visited, start);
19 }
```

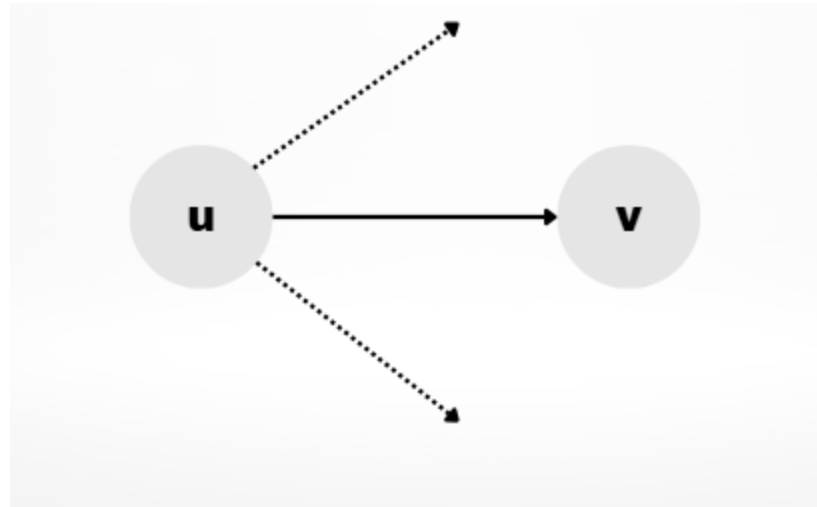
Time complexity

DFS

- A node is popped from stack.
- All of its neighbors are checked and non-visited nodes pushed into the stack.
- $O(V+E)$

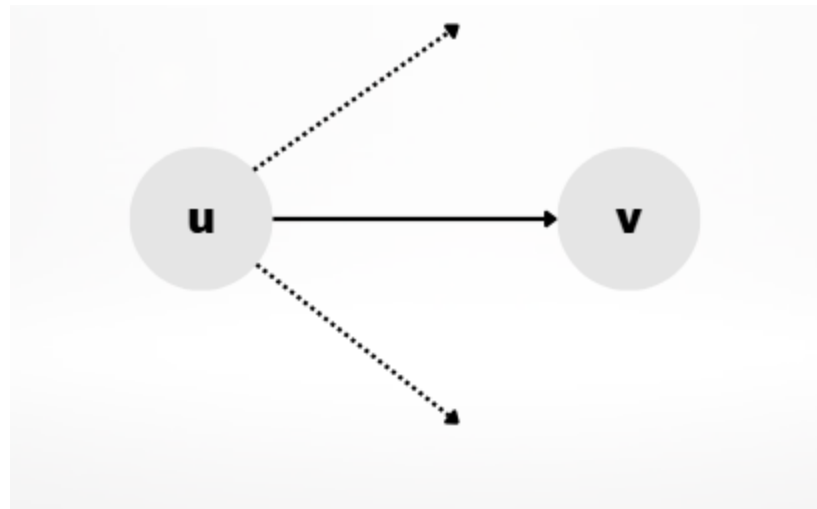
Topological Sorting

Given a directed acyclic graph $G = (V, E)$, topological sorting gives us a total ordering of G 's vertices such that for every edge (u, v) in E , vertex u precedes v in the ordering.



Topological Sorting

Given a directed acyclic graph $G = (V, E)$, topological sorting gives us a total ordering of G 's vertices such that for every edge (u, v) in E , vertex u precedes v in the ordering.



u comes before v in the ordering

Why do we need to Order A Graph?

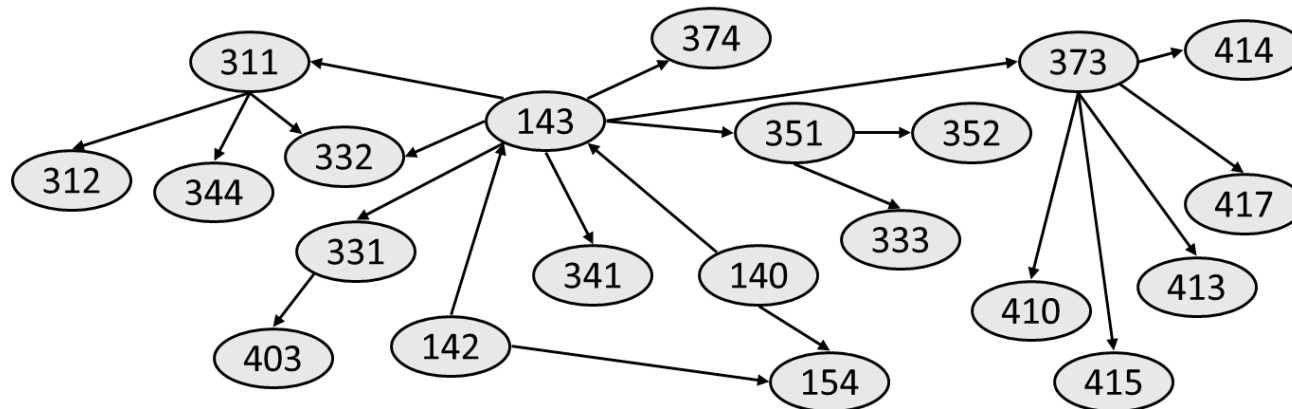
Many real-world problems require ordering tasks with dependencies.

Examples:

- Determining the order to recalculate updated cells in a spreadsheet
- Finding an order to recompile files that have dependencies

Why do we need to Order A Graph?

Suppose we have a directed acyclic graph (DAG) of courses, and we want to find an order in which the courses can be taken.



Topological Sorting

DFS style

```
6  stack<int> st;
7
8  void dfs(vector<vector<int>> &adj, vector<bool> &visited, int node) {
9      visited[node] = true;
10
11     for (int i : adj[node]) {
12         if (!visited[i]) {
13             dfs(adj, visited, i);
14         }
15     }
16
17     // push after visiting all adjacent nodes
18     st.push(node);
19 }
```

Topological Sorting

DFS style

```
21 void topologicalSort(vector<vector<int>> &adj) {
22     vector<bool> visited(adj.size(), false);
23
24     for (int i = 0; i < adj.size(); i++) {
25         if (!visited[i]) {
26             dfs(adj, visited, i);
27         }
28     }
29
30     while (!st.empty()) {
31         cout << st.top() << " ";
32         st.pop();
33     }
34 }
```


Time Complexity

TopSort: DFS style

```
21 void topologicalSort(vector<vector<int>> &adj) {
22     vector<bool> visited(adj.size(), false);
23
24     for (int i = 0; i < adj.size(); i++) {
25         if (!visited[i]) {
26             dfs(adj, visited, i);
27         }
28     }
29
30     while (!st.empty()) {
31         cout << st.top() << " ";
32         st.pop();
33     }
34 }
```

$O(V + E)$

$O(V)$

Time Complexity

TopSort: DFS style

```
21 void topologicalSort(vector<vector<int>> &adj) {
22     vector<bool> visited(adj.size(), false);
23
24     for (int i = 0; i < adj.size(); i++) {
25         if (!visited[i]) {
26             dfs(adj, visited, i);
27         }
28     }
29
30     while (!st.empty()) {
31         cout << st.top() << " ";
32         st.pop();
33     }
34 }
```

$O(V + E)$

$O(V)$

$= O(V + E)$